📘 RoBERTa-Based Two-Stage Ticket Classification System

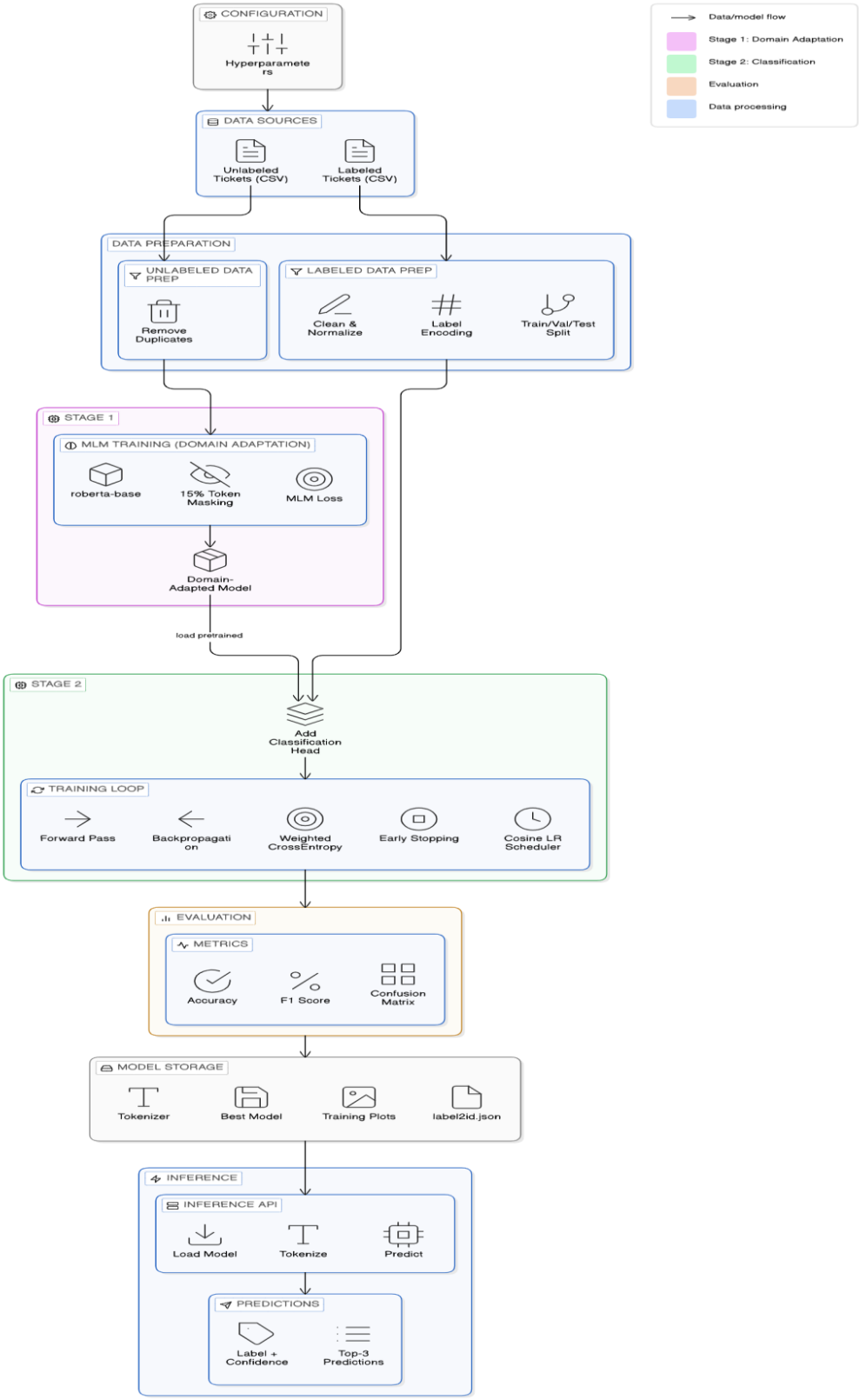
1. Project Overview

Modern enterprise service desks handle thousands of IT support tickets daily. These tickets must be categorized into domains such as access, network, software, hardware, and email. Manual classification is inefficient, error-prone, and introduces delays in issue resolution.

This project presents an automated ticket classification system using a two-stage transformer-based approach. The system leverages RoBERTa, a state-of-the-art NLP model, and improves its performance through domain-adaptive pretraining followed by supervised fine-tuning.

The primary objective is to build a robust, scalable, and production-ready classification pipeline that can accurately categorize tickets while handling noisy real-world data.

2. System Architecture



The system follows a **three-phase pipeline**:

Stage 0 — Data Preparation

- Multiple CSV datasets are merged
- Only ticket_description is extracted
- Data is cleaned and deduplicated
- Output: only_descriptions.csv

Stage 1 — Domain-Adaptive Pretraining

- Model: RobertaForMaskedLM
- Learns IT-specific language patterns from unlabeled data

Stage 2 — Classification Fine-Tuning

- Model: RobertaForSequenceClassification
- Uses Stage 1 weights for improved domain understanding

Inference Pipeline

Text → Tokenizer → RoBERTa → Softmax → Prediction

This architecture ensures that the model first **understands the domain** before performing classification.

---

3. Data Processing Pipeline

3.1 Data Sources

The system uses:

- Multiple CSV files for unlabeled data
- A labeled dataset containing:
    - ticket_description
    - ticket_category

3.2 Data Validation

The pipeline enforces strict validation:

- Only .csv files are accepted
- Missing columns trigger errors:
    - KeyError for missing fields
- Corrupt files raise parsing errors

This ensures **data integrity before training begins**.

3.3 Data Cleaning

The preprocessing pipeline includes:

- Removal of null values
- Stripping whitespace
- Lowercasing text and labels
- Removing empty rows
- Deduplication:
    - Exact duplicates removed
    - Duplicate descriptions removed

This improves data quality and reduces noise.

3.4 Unlabeled Data Preparation

For Stage 1:

- Multiple datasets are merged
- Only text is retained (no labels)
- Duplicates are removed
- Output is saved as:

only_descriptions.csv

This dataset acts as the **training corpus for domain adaptation**.

---

4. Stage 1 — Domain-Adaptive Pretraining (MLM)

4.1 Objective

To adapt a general-purpose RoBERTa model to the **IT service desk domain**.

4.2 Masked Language Modeling (MLM)

- 15% of tokens are randomly masked
- Model predicts masked words

Example:

Input : "VPN <mask> failing"
Target: "login"

This helps the model learn **contextual IT vocabulary**.

4.3 Training Setup

- Model: roberta-base
- Tokenizer: RoBERTa tokenizer
- Train/Validation split: 95% / 5%
- Dynamic masking using DataCollatorForLanguageModeling

4.4 Optimization Techniques

- **Learning Rate**: 5e-5
- **Scheduler**: Cosine decay
- **Warmup**: First 10% steps

- **Gradient Clipping**: max_norm = 1.0

These stabilize training and prevent divergence.

4.5 Evaluation Metrics

- **Loss**: Cross-entropy
- **Perplexity (PPL)**:

PPL = exp(loss)

Lower perplexity indicates better domain understanding.

4.6 Training Diagnostics

The system tracks:

- Loss reduction across epochs
- Trend indicators:
    - ↓ improving
    - ✗ stalled

4.7 Output Artifacts

- Fine-tuned RoBERTa model
- Tokenizer
- Configuration files

Saved to:

./roberta_it_stage1/

---

5. Stage 2 — Classification Fine-Tuning

5.1 Data Preparation

- Cleaned labeled dataset
- Labels normalized and encoded:

label2id = {"access": 0, "network": 1, ...}

5.2 Data Splitting

- Train: 80%
- Validation: 10%
- Test: 10%

Using **stratified sampling** to maintain class distribution.

5.3 Model Architecture

- Base: Stage 1 RoBERTa
- Head: Fully connected classification layer

Pipeline:

Text → Tokenizer → RoBERTa → Dense → Softmax

5.4 Loss Function

- **Weighted Cross-Entropy**
- **Label Smoothing (0.1)**

Benefits:

- Handles class imbalance
- Prevents overconfidence

5.5 Training Strategy

- Manual PyTorch training loop (not Trainer API)

- Key design decisions:
    - optimizer.zero_grad() before forward pass
    - Labels removed from batch to avoid double loss computation

---

5.6 Regularization

- **Early Stopping** (patience = 3)
- **Gradient Clipping**
- **Checkpointing best model**

---

6. Evaluation and Results

6.1 Metrics

- Accuracy
- Weighted F1 Score
- Macro F1 Score

6.2 Overfitting Analysis

Gap:

gap = val_loss - train_loss

Interpretation:

- < 0.05 → good fit
- 0.05–0.15 → mild overfit
- 0.15 → overfitting

6.3 Classification Report

Includes:

- Precision
- Recall
- F1-score per class

6.4 Confusion Matrix

Generated for:

- Train
- Validation
- Test

Both:

- Raw counts
- Normalized values

6.5 Learning Curves

Plots include:

- Loss
- Accuracy

---

7. Inference Pipeline

7.1 Prediction Flow

```
Input Text
  ↓
Tokenizer
  ↓
Model
  ↓
Softmax
```

↓
Prediction

7.2 Output Format

Each prediction returns:

- Predicted category
- Confidence score
- Entropy
- Margin

---

8. Testing and Validation

8.1 Test Coverage

- 114 test cases
- Covers:
    - Data ingestion
    - Preprocessing
    - Training
    - Evaluation

8.2 Mocking Strategy

- Transformer models mocked
- No GPU required
- Fast execution

8.3 Key Validations

- Input shape assertions
- File format checks
- Column validation
- Gradient flow verification

---

9. Conclusion

This project successfully implements a **robust and scalable ticket classification system** using a two-stage RoBERTa architecture.

Key strengths include:

- Domain adaptation for improved accuracy
- Strong data validation pipeline
- Production-grade training and evaluation

The system is suitable for **real-world enterprise deployment**, especially in IT service desk environments.

---

10. Future Work

- Hyperparameter tuning (Optuna)
- API deployment (FastAPI)
- Multi-label classification support
- Continuous Learning Pipeline

---

11. Limitations

Despite strong performance, the system has several practical limitations:

- Dependence on Labeled Data
- Sensitivity to Noisy or Ambiguous Input
- Computational Cost
- Fixed Sequence Length Constraint